

Problem 1

```
In [1]: import pandas as pd
import numpy as np
from scipy import optimize
import matplotlib.pyplot as plt

# Read the data
data = pd.read_csv("data.csv")
print(f"Data loaded, shape: {data.shape}")

# Group by search_id
search_groups = data.groupby('srch_id')

# Features to use in the MNL model
features = [
    'prop_starrating',
    'prop_review_score',
    'prop_brand_bool',
    'prop_location_score',
    'prop_accessibility_score',
    'prop_log_historical_price',
    'price_usd',
    'promotion_flag'
]

# Normalize features (as recommended by the professor)
# Save original data for later reference
data_original = data.copy()

# Store normalization parameters for later interpretation
normalization_params = {}

# Updated normalization code to include all features
for feature in features:
    # if feature not in ['prop_brand_bool', 'promotion_flag']: # Skip binary features
    mean = data[feature].mean()
    std = data[feature].std()
```

```

data[feature] = (data[feature] - mean) / std
normalization_params[feature] = {'mean': mean, 'std': std}
print(f"Normalized {feature}: Original mean={mean:.4f}, Original std={std:.4f}")

```

Function to calculate MNL Log-Likelihood

```
def mnl_log_likelihood(beta, data, search_groups, features):
```

Initialize log-likelihood

```
log_likelihood = 0
```

Extract beta values

```
beta0 = beta[0] # intercept
```

```
beta_features = beta[1:] # coefficients for features
```

Loop through each search session

```
for search_id, group in search_groups:
```

Get the hotels in this search

```
hotels = group.copy()
```

Calculate utilities

```
X = hotels[features]
```

```
utilities = beta0 + X.dot(beta_features)
```

```
hotels['utility'] = utilities
```

Calculate exp(utility) with numerical stability

```
max_utility = utilities.max()
```

```
hotels['exp_utility'] = np.exp(utilities - max_utility)
```

Calculate denominator (sum of exp(utility) + no-purchase option)

```
denominator = hotels['exp_utility'].sum() + np.exp(-max_utility) # no-purchase has utility 0
```

If there was a booking

```
if (hotels['booking_bool'] == 1).any():
```

Get the booked hotel

```
booked_hotel = hotels[hotels['booking_bool'] == 1].iloc[0]
```

Add to Log-Likelihood: Log(P(j|S))

```
log_likelihood += (booked_hotel['utility'] - max_utility) - np.log(denominator)
```

```
else:
```

No booking - customer chose outside option

```
log_likelihood += (-max_utility) - np.log(denominator)
```

```
return -log_likelihood # Negative because we're minimizing
```

```

# Function to fit MNL model
def fit_mnl_model(data, search_groups, features):
    # Initial beta values (all zeros)
    initial_beta = np.zeros(len(features) + 1) # +1 for intercept

    # Define the objective function for optimization
    def objective(beta):
        return mnl_log_likelihood(beta, data, search_groups, features)

    # Optimize using L-BFGS-B algorithm
    print("Starting optimization...")
    result = optimize.minimize(
        objective,
        initial_beta,
        method='L-BFGS-B',
        options={'disp': True, 'maxiter': 100}
    )

    print(f"Optimization completed: {result.success}")
    print(f"Final negative log-likelihood: {result.fun}")

    # Extract optimal beta values
    beta_opt = result.x

    return beta_opt

# Fit the MNL model with normalized features
beta_opt = fit_mnl_model(data, search_groups, features)

# Print results with proper interpretation accounting for normalization
print("\nMNL Model Parameters (with normalized features):")
print(f"Intercept ( $\beta_0$ ): {beta_opt[0]:.4f}")

feature_descriptions = {
    'prop_starrating': 'Star Rating (higher = better hotel quality)',
    'prop_review_score': 'Review Score (higher = better guest reviews)',
    'prop_brand_bool': 'Brand Flag (1 = branded hotel, 0 = independent)',
    'prop_location_score': 'Location Score (higher = better location)',
    'prop_accessibility_score': 'Accessibility Score (higher = better accessibility)',
    'prop_log_historical_price': 'Log Historical Price',
    'price_usd': 'Price in USD',
}

```

```

    'promotion_flag': 'Promotion Flag (1 = has promotion, 0 = no promotion)'
}

for i, feature in enumerate(features):
    coef = beta_opt[i+1]
    print(f"{feature_descriptions.get(feature, feature)} ( $\beta_{i+1}$ ): {coef:.6f}")

```

Data loaded, shape: (153009, 15)

Normalized prop_starrating: Original mean=3.1574, Original std=0.8572

Normalized prop_review_score: Original mean=3.9877, Original std=0.9071

Normalized prop_brand_bool: Original mean=0.7357, Original std=0.4409

Normalized prop_location_score: Original mean=2.6172, Original std=1.3482

Normalized prop_accessibility_score: Original mean=0.0060, Original std=0.0772

Normalized prop_log_historical_price: Original mean=4.2763, Original std=1.7895

Normalized price_usd: Original mean=141.1936, Original std=181.7699

Normalized promotion_flag: Original mean=0.1448, Original std=0.3519

Starting optimization...

Optimization completed: True

Final negative log-likelihood: 20611.325970651484

MNL Model Parameters (with normalized features):

Intercept (β_0): -1.7467

Star Rating (higher = better hotel quality) (β_1): 0.408127

Review Score (higher = better guest reviews) (β_2): 0.108755

Brand Flag (1 = branded hotel, 0 = independent) (β_3): 0.101379

Location Score (higher = better location) (β_4): 0.021997

Accessibility Score (higher = better accessibility) (β_5): 0.043445

Log Historical Price (β_6): -0.066872

Price in USD (β_7): -1.331102

Promotion Flag (1 = has promotion, 0 = no promotion) (β_8): 0.159767

In [4]: normalization_params_q1 = normalization_params.copy()

In [5]: normalization_params_q1

```
Out[5]: {'prop_starrating': {'mean': 3.157402505734957, 'std': 0.857185087363258},
        'prop_review_score': {'mean': 3.9876804632407246, 'std': 0.907122127197961},
        'prop_brand_bool': {'mean': 0.7357279637145527, 'std': 0.44094625279740235},
        'prop_location_score': {'mean': 2.6171728460417363, 'std': 1.348155303324557},
        'prop_accessibility_score': {'mean': 0.005993111516316034,
        'std': 0.07718311385622265},
        'prop_log_historical_price': {'mean': 4.27631054382422,
        'std': 1.7895092329050903},
        'price_usd': {'mean': 141.19362259736354, 'std': 181.76994386241384},
        'promotion_flag': {'mean': 0.14480847531844532, 'std': 0.3519087810174338}}
```

Problem 2

Assumption:

- We use the β from problem 1

```
betas = [-1.7467, 0.408127, 0.108755, 0.101379, 0.021997, 0.043445, -0.066872, -1.331102,
0.159767]
```

- The only identifier for all the hotels in `data1.csv` is price

```
In [12]: import pandas as pd
import numpy as np

betas = [-1.7467, 0.408127, 0.108755, 0.101379, 0.021997, 0.043445, -0.066872, -1.331102, 0.159767]

def process_data( data_filepath):
    """
    Process data1.csv in the same way as data.csv
    Inputs:
        data_filepath (str): file path for data1.csv, data2.csv...
    Outputs:
        normalized_data (DataFrame): the normalized data1.csv, data2.csv ...
    """
    df_norm_params = pd.read_csv("normalization_params_q1.csv")
```

```

df_data = pd.read_csv(data_filepath)
columns = df_norm_params.columns
for col in columns:
    mean = df_norm_params.loc[0, col] # mean
    std = df_norm_params.loc[1, col] # std
    df_data[col] = (df_data[col] - mean) / std
return df_data
def compute_preference_weight(betas, assortment):
    """
    Inputs:
        betas (List[int]): a list of 9 betas, include beta0
        assortment (DataFrame)
    Outputs:
        pref_weight (List[float]): a list of v_j for all the hotel in the assortment S
    """
    pref_weights=[]
    for j, row in assortment.iterrows():
        hotel_features = row.tolist() # a list of x_ji values
        hotel_features.insert(0,1) # augmented by 1
        u_j = np.dot(hotel_features, betas)
        v_j = np.exp(u_j)
        pref_weights.append(v_j)
    return pref_weights

def expected_rev(assortment,betas):
    """
    Inputs:
        assortment (DataFrame), NORMALIZED DATA VALUES
    Outputs:
        expected_rev (float)
    """
    expected_rev = 0
    pref_weights = compute_preference_weight(betas, assortment)
    # print(f"pref_weights = {pref_weights}")
    # print(f"pref_weights: {len(pref_weights)}, shape of the assortment: {assortment.shape}")
    j = 0
    total_prob = 0
    for _,row in assortment.iterrows():
        pj = row["price_usd"] * 181.76994386241384 + 141.19362259736354 # hard coding
        # choice_prob = pref_weights[j]/( np.exp(betas[0]) +np.sum(pref_weights))
        choice_prob = pref_weights[j]/( 1+np.sum(pref_weights))
        expected_rev += pj * choice_prob

```

```

        # print(f" j = {j}, choix_prob is {choice_prob}")
        total_prob += choice_prob
        j+=1
    # print("expected_rev =", expected_rev, "total_prob=",total_prob)
    return expected_rev

def optim_assortment(df, betas):
    """
    Inputs:
        df (DataFrame): all hotels and features, NORMALIZED DATA VALUES
        pref_weights (List[float]): a list of v_j for all the hotels in the assortment
    Outputs:
        sorted_assortment_rev (Dict[Str:Float]): a dict of assortment with revenue sorted by revenue

    """
    # sort by price
    df_sorted = df.sort_values(by="price_usd", ascending=False)
    assortment_rev = dict() # {"j": rev}
    idx=1

    for _, row in df_sorted.iterrows():
        # each hotel j
        new_assortment = df_sorted.nlargest(idx, "price_usd")
        exp_rev = expected_rev(new_assortment, betas)
        assortment_rev[str(idx)] = exp_rev
        # print(f"the expected_rev for idx={idx} is {exp_rev}")
        idx+=1

    sorted_assortment_rev = dict(sorted(assortment_rev.items(), key=lambda x: x[1], reverse=True))
    opt_assortment, opt_rev = next(iter(sorted_assortment_rev.items()))
    print(f"include the first {opt_assortment} hotels, the optimal revenue is {opt_rev}")
    return sorted_assortment_rev

# compute the optimal assortment for each doc

print("For data1.csv")
data1_df = process_data("data1.csv")
opt_ass1 = optim_assortment(data1_df, betas)

print("For data2.csv")
data2_df = process_data('data2.csv')

```

```

opt_ass2 = optim_assortment(data2_df, betas)

print("For data3.csv")
data3_df = process_data("data3.csv")
opt_ass3 = optim_assortment(data3_df, betas)
print("For data4.csv")
data4_df = process_data("data4.csv")
opt_ass4 = optim_assortment(data4_df, betas)

```

For data1.csv
include the first 18 hotels, the optimal revenue is 107.35153015966996

For data2.csv
include the first 10 hotels, the optimal revenue is 131.1103399233331

For data3.csv
include the first 18 hotels, the optimal revenue is 121.05257044249225

For data4.csv
include the first 11 hotels, the optimal revenue is 97.40757326301659

Unit test for problem 2

```

In [23]: import pandas as pd
betas = [-1.7467, 0.408127, 0.108755, 0.101379, 0.021997, 0.043445, -0.066872, -1.331102, 0.159767]

df_sorted = pd.read_csv("data1.csv").sort_values(by="price_usd", ascending=False)
df_sorted.to_csv("sorted_data1.csv", index=False)

df_v1 = process_data("sorted_data1_v1.csv")
df_v2 = process_data("sorted_data1_v2.csv")

```

```

In [ ]: def test_q2(df, betas):
        """
        Inputs:
            df (DataFrame): all hotels and features
            pref_weights (List[float]): a list of v_j for all the hotels in the assortment
        Outputs:
            sorted_assortment_rev (Dict[Str:Float]): a dict of assortment with revenue sorted by revenue

        """
        # sort by price
        df_sorted = df.sort_values(by="price_usd", ascending=False)
        assortment_rev = dict() # {"j": rev}

```

```

idx=1

for _, row in df_sorted.iterrows():
    # each hotel j
    new_assortment = df_sorted.nlargest(idx, "price_usd")
    exp_rev = expected_rev(new_assortment, betas)
    assortment_rev[str(idx)] = exp_rev
    # print(f"the expected_rev for idx={idx} is {exp_rev}")
    idx+=1

sorted_assortment_rev = dict(sorted(assortment_rev.items(), key=lambda x: x[1], reverse=True))
opt_assortment, opt_rev = next(iter(sorted_assortment_rev.items()))
print(f"include the first {opt_assortment} hotels, the optimal revenue is {opt_rev}")
return sorted_assortment_rev

exp_rev_v1 = expected_rev(df_v1,betas)
exp_rev_v2 = expected_rev(df_v2,betas)
print(f"For sorted_data1_v1, the exp rev= {exp_rev_v1}")
print(f"For sorted_data1_v2, the exp rev= {exp_rev_v2}")
# df1 = process_data("sorted_data1.csv")
# print("exp rev for optimal assortment", expected_rev(df1,betas))

```

For sorted_data1_v1, the exp rev= 107.34141181120337
 For sorted_data1_v2, the exp rev= 107.34380723603107
 exp rev for optimal assortment 107.35153015966996

Problem 3

Assumptions:

- provide all the hotels in data1.csv to customers.

Expected revenue if the customer is provided by assortment 1:

$$R(S_1) = \sum_{j \in S_1} p_j P(j|S_1) = \sum_{j \in S} p_j \frac{e^{\beta_0 + \sum_{i=1,..,s} \beta_i x_j^i}}{1 + \sum_{i \in S} e^{\beta_i x_j^i}}$$

$\backslash \text{argmax} R(S)$

```

In [20]: import pandas as pd
import numpy as np
from scipy.optimize import minimize

def neg_expected_revenue_mnl(prices, filename):
    """
    Computes the negative expected revenue under MNL model given hotel prices.
    Inputs:
        prices (np.array): NOT normalized hotel prices (num_hotels,)
        filename (str): CSV filename containing hotel data
    Outputs:
        Negative expected revenue (scalar)
    """
    df = process_data(filename) # use your custom preprocessor
    num_hotels = df.shape[0]

    betas = np.array([-1.7467, 0.408127, 0.108755, 0.101379, 0.021997, 0.043445, -0.066872, -1.331102, 0.159767])

    features = df[['prop_starrating', 'prop_review_score', 'prop_brand_bool',
                   'prop_location_score', 'prop_accessibility_score',
                   'prop_log_historical_price', 'price_usd', 'promotion_flag']].to_numpy()
    norm_prices = (prices - 141.19362259736354) / 181.76994386241384
    X = np.hstack((
        np.ones((num_hotels, 1)),          # Intercept
        features[:, :6],                   # Features 1-6
        norm_prices.reshape(-1, 1),        # Feature 7: normalized price
        features[:, 7].reshape(-1, 1)      # Feature 8: promotion flag
    ))
    utilities = X @ betas
    exp_utilities = np.exp(utilities)
    denom = 1 + np.sum(exp_utilities)
    choice_probs = exp_utilities / denom

    revenue = np.sum(prices * choice_probs)
    return -revenue

def optimize_prices(filename):
    """
    Optimizes hotel prices for the given CSV file using MNL model.
    """

```

```

df = process_data(filename)
initial_prices = df['price_usd'].to_numpy()
bounds = [(20, 500)] * len(initial_prices)

result = minimize(
    fun=lambda p: neg_expected_revenue_mnl(p, filename),
    x0=initial_prices,
    bounds=bounds,
    method='L-BFGS-B'
)

if result.success:
    optimal_prices = result.x
    print(f"Optimization succeeded for {filename}")
    print("Optimal prices (sorted):", list(np.round(optimal_prices, 2)))
    print("Max expected revenue:", -result.fun)
else:
    print(f"Optimization failed for {filename}")
    print(result.message)

```

```

In [21]: optimize_prices('data1.csv')
optimize_prices('data2.csv')
optimize_prices('data3.csv')
optimize_prices('data4.csv')

```

Optimization succeeded for data1.csv

Optimal prices (sorted): [314.35, 314.34, 314.36, 314.36, 314.36, 314.35, 314.36, 314.4, 314.29, 314.42, 314.36, 314.4, 314.42, 314.38, 314.38, 314.38, 314.41, 314.34, 314.35, 314.35, 314.36, 314.38, 314.37, 314.37, 314.35, 314.35, 314.36]

Max expected revenue: 177.80141982576177

Optimization succeeded for data2.csv

Optimal prices (sorted): [385.35, 385.35, 385.36, 385.36, 385.35, 385.34, 385.37, 385.36, 385.35, 385.35, 385.35, 385.34, 385.35, 385.32, 385.3, 385.35, 385.29, 385.34, 385.32, 385.35, 385.34, 385.34, 385.35, 385.3, 385.34, 385.36, 385.34, 385.36, 385.34]

Max expected revenue: 248.79496490716548

Optimization succeeded for data3.csv

Optimal prices (sorted): [313.0, 313.01, 313.01, 313.03, 313.01, 313.01, 313.01, 313.0, 313.01, 312.97, 313.0, 312.99, 313.01, 313.01, 313.01, 313.02, 313.0, 313.0, 313.02, 313.01, 313.02, 312.97, 312.99, 313.03, 312.99, 313.01]

Max expected revenue: 176.46827350187695

Optimization succeeded for data4.csv

Optimal prices (sorted): [351.03, 351.03, 351.05, 350.98, 351.02, 351.0, 351.02, 351.0, 351.03, 351.02, 351.02, 351.02, 351.05, 351.02, 351.01, 351.01, 350.98, 351.01, 350.96, 351.02, 351.02, 351.03, 351.02, 350.99, 351.01, 351.02, 351.02, 351.02]

Max expected revenue: 214.46889658204097

Problem 4

```
In [ ]: import pandas as pd
import numpy as np
from scipy import optimize
import matplotlib.pyplot as plt

# Read the data
data = pd.read_csv("data.csv")
print(f"Data loaded, shape: {data.shape}")

# Define customer segments based on booking window
data['customer_segment'] = np.where(data['srch_booking_window'] < 7, 2, 1) # 2 for late, 1 for early

# Calculate theta values (probability of customer belonging to each segment)
unique_searches = data['srch_id'].nunique()
# Count unique searches in each segment (we need to count searches, not individual hotel options)
segment_counts = data.drop_duplicates(subset=['srch_id'])[['srch_id', 'customer_segment']].groupby('customer_segment').count()
theta_1 = segment_counts.loc[1, 'srch_id'] / unique_searches # early bookings
theta_2 = segment_counts.loc[2, 'srch_id'] / unique_searches # late bookings
```

```

print(f"\nTotal unique searches: {unique_searches}")
print(f"Early booking searches (segment 1): {segment_counts.loc[1, 'srch_id']}")
print(f"Late booking searches (segment 2): {segment_counts.loc[2, 'srch_id']}")
print(f" $\theta_1$  (probability of early booking): {theta_1:.4f}")
print(f" $\theta_2$  (probability of late booking): {theta_2:.4f}")

# Features to use in the MNL model
features = [
    'prop_starrating',
    'prop_review_score',
    'prop_brand_bool',
    'prop_location_score',
    'prop_accessibility_score',
    'prop_log_historical_price',
    'price_usd',
    'promotion_flag'
]

# Create separate datasets for each segment
data_segment_1 = data[data['customer_segment'] == 1].copy() # early bookings
data_segment_2 = data[data['customer_segment'] == 2].copy() # late bookings

# Group by search_id for each segment
search_groups_1 = data_segment_1.groupby('srch_id')
search_groups_2 = data_segment_2.groupby('srch_id')

# Store original data for each segment (before normalization)
data_original_1 = data_segment_1.copy()
data_original_2 = data_segment_2.copy()

# Normalize features together in both segments - including booleans
'''
normalization_params_1 = {}
print("\nNormalizing features for Segment 1 (Early Bookings):")
for feature in features:
    mean = data_segment_1[feature].mean()
    std = data_segment_1[feature].std()
    data_segment_1[feature] = (data_segment_1[feature] - mean) / std
    normalization_params_1[feature] = {'mean': mean, 'std': std}
    print(f"  Normalized {feature}: mean={mean:.4f}, std={std:.4f}")

```

```

normalization_params_2 = {}
print("\nNormalizing features for Segment 2 (Late Bookings):")
for feature in features:
    mean = data_segment_2[feature].mean()
    std = data_segment_2[feature].std()
    data_segment_2[feature] = (data_segment_2[feature] - mean) / std
    normalization_params_2[feature] = {'mean': mean, 'std': std}
    print(f"   Normalized {feature}: mean={mean:.4f}, std={std:.4f}")
...

# Apply normalization using normalization_params_q1 to Segment 1
print("\nApplying normalization_params_q1 to Segment 1 (Early Bookings):")
for feature in features:
    mean_q1 = normalization_params_q1[feature]['mean']
    std_q1 = normalization_params_q1[feature]['std']
    data_segment_1[feature] = (data_segment_1[feature] - mean_q1) / std_q1
    segment1_mean = data_segment_1[feature].mean()
    segment1_std = data_segment_1[feature].std()
    print(f"   Normalized {feature}: new mean={segment1_mean:.4f}, new std={segment1_std:.4f}")

# Apply normalization using normalization_params_q1 to Segment 2
print("\nApplying normalization_params_q1 to Segment 2 (Late Bookings):")
for feature in features:
    mean_q1 = normalization_params_q1[feature]['mean']
    std_q1 = normalization_params_q1[feature]['std']
    data_segment_2[feature] = (data_segment_2[feature] - mean_q1) / std_q1
    segment2_mean = data_segment_2[feature].mean()
    segment2_std = data_segment_2[feature].std()
    print(f"   Normalized {feature}: new mean={segment2_mean:.4f}, new std={segment2_std:.4f}")

# Function to calculate MNL log-likelihood (same as before)
def mnl_log_likelihood(beta, data, search_groups, features):
    # Initialize log-likelihood
    log_likelihood = 0

    # Extract beta values
    beta0 = beta[0] # intercept
    beta_features = beta[1:] # coefficients for features

    # Loop through each search session
    for search_id, group in search_groups:
        # Get the hotels in this search
        hotels = group.copy()

```

```

    # Calculate utilities
    X = hotels[features]
    utilities = beta0 + X.dot(beta_features)
    hotels['utility'] = utilities

    # Calculate exp(utility) with numerical stability
    max_utility = utilities.max()
    hotels['exp_utility'] = np.exp(utilities - max_utility)

    # Calculate denominator (sum of exp(utility) + no-purchase option)
    denominator = hotels['exp_utility'].sum() + np.exp(-max_utility) # no-purchase has utility 0

    # If there was a booking
    if (hotels['booking_bool'] == 1).any():
        # Get the booked hotel
        booked_hotel = hotels[hotels['booking_bool'] == 1].iloc[0]

        # Add to Log-likelihood:  $\log(P(j|S))$ 
        log_likelihood += (booked_hotel['utility'] - max_utility) - np.log(denominator)
    else:
        # No booking - customer chose outside option
        log_likelihood += (-max_utility) - np.log(denominator)

    return -log_likelihood # Negative because we're minimizing

# Function to fit MNL model
def fit_mnl_model(data, search_groups, features, segment_name):
    # Initial beta values (all zeros)
    initial_beta = np.zeros(len(features) + 1) # +1 for intercept

    # Define the objective function for optimization
    def objective(beta):
        return mnl_log_likelihood(beta, data, search_groups, features)

    # Optimize using L-BFGS-B algorithm
    print(f"\nStarting optimization for {segment_name}...")
    result = optimize.minimize(
        objective,
        initial_beta,
        method='L-BFGS-B',
        options={'disp': True, 'maxiter': 100}
    )

```

```

)

print(f"Optimization completed for {segment_name}: {result.success}")
print(f"Final negative log-likelihood: {result.fun}")

# Extract optimal beta values
beta_opt = result.x

return beta_opt

# Fit MNL model for each segment
beta_opt_1 = fit_mnl_model(data_segment_1, search_groups_1, features, "Segment 1 (Early Bookings)")
beta_opt_2 = fit_mnl_model(data_segment_2, search_groups_2, features, "Segment 2 (Late Bookings)")

# Helper function to print results
def print_mnl_results(beta_opt, segment_name, normalization_params):
    print(f"\nMNL Model Parameters for {segment_name}:")
    print(f"Intercept ( $\beta_0$ ): {beta_opt[0]:.4f}")

    feature_descriptions = {
        'prop_starrating': 'Star Rating (higher = better hotel quality)',
        'prop_review_score': 'Review Score (higher = better guest reviews)',
        'prop_brand_bool': 'Brand Flag (1 = branded hotel, 0 = independent)',
        'prop_location_score': 'Location Score (higher = better location)',
        'prop_accessibility_score': 'Accessibility Score (higher = better accessibility)',
        'prop_log_historical_price': 'Log Historical Price',
        'price_usd': 'Price in USD',
        'promotion_flag': 'Promotion Flag (1 = has promotion, 0 = no promotion)'
    }

    # Store coefficients for comparison
    coefficients = []
    feature_names = []

    for i, feature in enumerate(features):
        coef = beta_opt[i+1]
        print(f"{feature_descriptions.get(feature, feature)} ( $\beta_{i+1}$ ): {coef:.6f}")
        coefficients.append(coef)
        feature_names.append(feature)
    ...

# Additional interpretation of the coefficients (considering normalization)
print("\nInterpretation of Coefficients:")

```

```

for i, feature in enumerate(features):
    coef = beta_opt[i+1]
    if feature in normalization_params:
        # For normalized features, we need to interpret differently
        std = normalization_params[feature]['std']
        original_coef = coef / std
        print(f"{feature}: A one-unit increase in the original {feature} increases utility by {original_coef:.6f}")
    else:
        # For binary features, interpretation remains the same
        print(f"{feature}: Presence of this feature changes utility by {coef:.6f}")

'''

return coefficients, feature_names

# Print results for each segment
coef_1, features_list = print_mnl_results(beta_opt_1, "Segment 1 (Early Bookings)", normalization_params_1)
coef_2, _ = print_mnl_results(beta_opt_2, "Segment 2 (Late Bookings)", normalization_params_2)

# Compare coefficients between segments
print("\nComparison of Coefficients between Segments:")
print("Feature | Early Booking ( $\beta$ ) | Late Booking ( $\beta$ ) | Difference")
print("-----|-----|-----|-----")
for i, feature in enumerate(features_list):
    diff = coef_1[i] - coef_2[i]
    print(f"{feature} | {coef_1[i]:.6f} | {coef_2[i]:.6f} | {diff:.6f}")

```

Data loaded, shape: (153009, 15)

Total unique searches: 8354

Early booking searches (segment 1): 4537

Late booking searches (segment 2): 3817

θ_1 (probability of early booking): 0.5431

θ_2 (probability of late booking): 0.4569

Applying normalization_params_q1 to Segment 1 (Early Bookings):

Normalized prop_starrating: new mean=0.0623, new std=1.0079

Normalized prop_review_score: new mean=0.0184, new std=0.9901

Normalized prop_brand_bool: new mean=-0.0489, new std=1.0246

Normalized prop_location_score: new mean=0.0407, new std=1.0167

Normalized prop_accessibility_score: new mean=0.0035, new std=1.0219

Normalized prop_log_historical_price: new mean=0.0022, new std=1.0191

Normalized price_usd: new mean=0.0361, new std=1.2242

Normalized promotion_flag: new mean=0.0471, new std=1.0454

Applying normalization_params_q1 to Segment 2 (Late Bookings):

Normalized prop_starrating: new mean=-0.0756, new std=0.9850

Normalized prop_review_score: new mean=-0.0224, new std=1.0115

Normalized prop_brand_bool: new mean=0.0593, new std=0.9660

Normalized prop_location_score: new mean=-0.0493, new std=0.9771

Normalized prop_accessibility_score: new mean=-0.0042, new std=0.9728

Normalized prop_log_historical_price: new mean=-0.0027, new std=0.9763

Normalized price_usd: new mean=-0.0437, new std=0.6259

Normalized promotion_flag: new mean=-0.0571, new std=0.9388

Starting optimization for Segment 1 (Early Bookings)...

Optimization completed for Segment 1 (Early Bookings): True

Final negative log-likelihood: 11052.587145050955

Starting optimization for Segment 2 (Late Bookings)...

Optimization completed for Segment 2 (Late Bookings): True

Final negative log-likelihood: 9505.447785059454

MNL Model Parameters for Segment 1 (Early Bookings):

Intercept (β_0): -1.9180

Star Rating (higher = better hotel quality) (β_1): 0.379038

Review Score (higher = better guest reviews) (β_2): 0.124630

Brand Flag (1 = branded hotel, 0 = independent) (β_3): 0.092098

Location Score (higher = better location) (β_4): -0.021459

Accessibility Score (higher = better accessibility) (β_5): 0.057844
Log Historical Price (β_6): -0.094884
Price in USD (β_7): -1.071555
Promotion Flag (1 = has promotion, 0 = no promotion) (β_8): 0.134802

MNL Model Parameters for Segment 2 (Late Bookings):

Intercept (β_0): -1.5397
Star Rating (higher = better hotel quality) (β_1): 0.462759
Review Score (higher = better guest reviews) (β_2): 0.095586
Brand Flag (1 = branded hotel, 0 = independent) (β_3): 0.112906
Location Score (higher = better location) (β_4): 0.087280
Accessibility Score (higher = better accessibility) (β_5): 0.025559
Log Historical Price (β_6): -0.031981
Price in USD (β_7): -1.696923
Promotion Flag (1 = has promotion, 0 = no promotion) (β_8): 0.194143

Comparison of Coefficients between Segments:

Feature	Early Booking (β)	Late Booking (β)	Difference
prop_starrating	0.379038	0.462759	-0.083721
prop_review_score	0.124630	0.095586	0.029044
prop_brand_bool	0.092098	0.112906	-0.020808
prop_location_score	-0.021459	0.087280	-0.108738
prop_accessibility_score	0.057844	0.025559	0.032285
prop_log_historical_price	-0.094884	-0.031981	-0.062903
price_usd	-1.071555	-1.696923	0.625368
promotion_flag	0.134802	0.194143	-0.059341

Problem 5

```
In [20]: import pandas as pd

df_norm_params = pd.DataFrame(normalization_params_q1).T
df_norm_params = df_norm_params.T

# Save to CSV
df_norm_params.to_csv("normalization_params_q1.csv", index=False)
```

refer to "Question5.ipynb"

Problem 6

```
In [7]: import pandas as pd
import numpy as np
from scipy import optimize
import matplotlib.pyplot as plt

# Read the data
data = pd.read_csv("data.csv")
print(f"Data loaded, shape: {data.shape}")

# Define customer segments based on booking window
data['customer_segment'] = np.where(data['srch_saturday_night_bool'] == 1, 2, 1) # 2 for sat night, 1 for non sat night

# Calculate theta values (probability of customer belonging to each segment)
unique_searches = data['srch_id'].nunique()
# Count unique searches in each segment (we need to count searches, not individual hotel options)
segment_counts = data.drop_duplicates(subset=['srch_id'])[['srch_id', 'customer_segment']].groupby('customer_segment').count()
theta_1 = segment_counts.loc[1, 'srch_id'] / unique_searches # non sat night
theta_2 = segment_counts.loc[2, 'srch_id'] / unique_searches # sat night

print(f"\nTotal unique searches: {unique_searches}")
print(f"Non-Saturday searches (segment 1): {segment_counts.loc[1, 'srch_id']}")
print(f"Saturday searches (segment 2): {segment_counts.loc[2, 'srch_id']}")
print(f" $\theta_1$  (probability of non-Sat search): {theta_1:.4f}")
print(f" $\theta_2$  (probability of Sat search): {theta_2:.4f}")

# Features to use in the MNL model
features = [
    'prop_starrating',
    'prop_review_score',
    'prop_brand_bool',
    'prop_location_score',
    'prop_accessibility_score',
    'prop_log_historical_price',
    'price_usd',
    'promotion_flag'
]
```

```

# Create separate datasets for each segment
data_segment_1 = data[data['customer_segment'] == 1].copy() # non Sat night
data_segment_2 = data[data['customer_segment'] == 2].copy() # Sat night

# Group by search_id for each segment
search_groups_1 = data_segment_1.groupby('srch_id')
search_groups_2 = data_segment_2.groupby('srch_id')

# Store original data for each segment (before normalization)
data_original_1 = data_segment_1.copy()
data_original_2 = data_segment_2.copy()

# Normalize features together in both segments
# Apply normalization using normalization_params_q1 to Segment 1
print("\nApplying normalization_params_q1 to Segment 1 (Non-Sat Bookings):")
for feature in features:
    mean_q1 = normalization_params_q1[feature]['mean']
    std_q1 = normalization_params_q1[feature]['std']
    data_segment_1[feature] = (data_segment_1[feature] - mean_q1) / std_q1
    segment1_mean = data_segment_1[feature].mean()
    segment1_std = data_segment_1[feature].std()
    print(f"    Normalized {feature}: new mean={segment1_mean:.4f}, new std={segment1_std:.4f}")

# Apply normalization using normalization_params_q1 to Segment 2
print("\nApplying normalization_params_q1 to Segment 2 (Sat Bookings):")
for feature in features:
    mean_q1 = normalization_params_q1[feature]['mean']
    std_q1 = normalization_params_q1[feature]['std']
    data_segment_2[feature] = (data_segment_2[feature] - mean_q1) / std_q1
    segment2_mean = data_segment_2[feature].mean()
    segment2_std = data_segment_2[feature].std()
    print(f"    Normalized {feature}: new mean={segment2_mean:.4f}, new std={segment2_std:.4f}")

# Function to calculate MNL log-Likelihood (same as before)
def mnl_log_likelihood(beta, data, search_groups, features):
    # Initialize log-likelihood
    log_likelihood = 0

    # Extract beta values
    beta0 = beta[0] # intercept
    beta_features = beta[1:] # coefficients for features

```

```

# Loop through each search session
for search_id, group in search_groups:
    # Get the hotels in this search
    hotels = group.copy()

    # Calculate utilities
    X = hotels[features]
    utilities = beta0 + X.dot(beta_features)
    hotels['utility'] = utilities

    # Calculate exp(utility) with numerical stability
    max_utility = utilities.max()
    hotels['exp_utility'] = np.exp(utilities - max_utility)

    # Calculate denominator (sum of exp(utility) + no-purchase option)
    denominator = hotels['exp_utility'].sum() + np.exp(-max_utility) # no-purchase has utility 0

    # If there was a booking
    if (hotels['booking_bool'] == 1).any():
        # Get the booked hotel
        booked_hotel = hotels[hotels['booking_bool'] == 1].iloc[0]

        # Add to Log-likelihood: Log(P(j|S))
        log_likelihood += (booked_hotel['utility'] - max_utility) - np.log(denominator)
    else:
        # No booking - customer chose outside option
        log_likelihood += (-max_utility) - np.log(denominator)

    return -log_likelihood # Negative because we're minimizing

# Function to fit MNL model
def fit_mnl_model(data, search_groups, features, segment_name):
    # Initial beta values (all zeros)
    initial_beta = np.zeros(len(features) + 1) # +1 for intercept

    # Define the objective function for optimization
    def objective(beta):
        return mnl_log_likelihood(beta, data, search_groups, features)

    # Optimize using L-BFGS-B algorithm
    print(f"\nStarting optimization for {segment_name}...")
    result = optimize.minimize(

```

```

    objective,
    initial_beta,
    method='L-BFGS-B',
    options={'disp': True, 'maxiter': 100}
)

print(f"Optimization completed for {segment_name}: {result.success}")
print(f"Final negative log-likelihood: {result.fun}")

# Extract optimal beta values
beta_opt = result.x

return beta_opt

```

Data loaded, shape: (153009, 15)

Total unique searches: 8354

Non-Saturday searches (segment 1): 3537

Saturday searches (segment 2): 4817

θ_1 (probability of non-Sat search): 0.4234

θ_2 (probability of Sat search): 0.5766

Applying normalization_params_q1 to Segment 1 (Non-Sat Bookings):

Normalized prop_starrating: new mean=0.0372, new std=1.0068

Normalized prop_review_score: new mean=-0.0035, new std=0.9916

Normalized prop_brand_bool: new mean=-0.0370, new std=1.0189

Normalized prop_location_score: new mean=0.0215, new std=1.0129

Normalized prop_accessibility_score: new mean=-0.0029, new std=0.9813

Normalized prop_log_historical_price: new mean=-0.0130, new std=1.0165

Normalized price_usd: new mean=-0.0020, new std=0.6541

Normalized promotion_flag: new mean=0.0134, new std=1.0133

Applying normalization_params_q1 to Segment 2 (Sat Bookings):

Normalized prop_starrating: new mean=-0.0281, new std=0.9939

Normalized prop_review_score: new mean=0.0026, new std=1.0063

Normalized prop_brand_bool: new mean=0.0280, new std=0.9845

Normalized prop_location_score: new mean=-0.0162, new std=0.9899

Normalized prop_accessibility_score: new mean=0.0022, new std=1.0139

Normalized prop_log_historical_price: new mean=0.0098, new std=0.9872

Normalized price_usd: new mean=0.0015, new std=1.1968

Normalized promotion_flag: new mean=-0.0101, new std=0.9897

```

In [ ]: # Fit MNL model for each segment
beta_opt_1 = fit_mnl_model(data_segment_1, search_groups_1, features, "Segment 1 (Non-Sat Bookings)")
beta_opt_2 = fit_mnl_model(data_segment_2, search_groups_2, features, "Segment 2 (Sat Bookings)")

# Helper function to print results
def print_mnl_results(beta_opt, segment_name, normalization_params):
    print(f"\nMNL Model Parameters for {segment_name}:")
    print(f"Intercept ( $\beta_0$ ): {beta_opt[0]:.4f}")

    feature_descriptions = {
        'prop_starrating': 'Star Rating (higher = better hotel quality)',
        'prop_review_score': 'Review Score (higher = better guest reviews)',
        'prop_brand_bool': 'Brand Flag (1 = branded hotel, 0 = independent)',
        'prop_location_score': 'Location Score (higher = better location)',
        'prop_accessibility_score': 'Accessibility Score (higher = better accessibility)',
        'prop_log_historical_price': 'Log Historical Price',
        'price_usd': 'Price in USD',
        'promotion_flag': 'Promotion Flag (1 = has promotion, 0 = no promotion)'
    }

    # Store coefficients for comparison
    coefficients = []
    feature_names = []

    for i, feature in enumerate(features):
        coef = beta_opt[i+1]
        print(f"{feature_descriptions.get(feature, feature)} ( $\beta_{i+1}$ ): {coef:.6f}")
        coefficients.append(coef)
        feature_names.append(feature)
    ...

# Additional interpretation of the coefficients (considering normalization)
print("\nInterpretation of Coefficients:")
for i, feature in enumerate(features):
    coef = beta_opt[i+1]
    if feature in normalization_params:
        # For normalized features, we need to interpret differently
        std = normalization_params[feature]['std']
        original_coef = coef / std
        print(f"{feature}: A one-unit increase in the original {feature} increases utility by {original_coef:.6f}")
    else:
        # For binary features, interpretation remains the same

```

```

        print(f"{feature}: Presence of this feature changes utility by {coef:.6f}")

    ...

    return coefficients, feature_names

```

Starting optimization for Segment 1 (Non-Sat Bookings)...

Optimization completed for Segment 1 (Non-Sat Bookings): True

Final negative log-likelihood: 8775.771219710332

Starting optimization for Segment 2 (Sat Bookings)...

Optimization completed for Segment 2 (Sat Bookings): True

Final negative log-likelihood: 11827.069625491831

```

In [9]: # Print results for each segment
coef_1, features_list = print_mnl_results(beta_opt_1, "Segment 1 (Non-Sat Bookings)", normalization_params)
coef_2, _ = print_mnl_results(beta_opt_2, "Segment 2 (Sat Bookings)", normalization_params)

# Compare coefficients between segments
print("\nComparison of Coefficients between Segments:")
print("Feature | Non-Sat Booking ( $\beta$ ) | Sat Booking ( $\beta$ ) | Difference")
print("-----|-----|-----|-----")
for i, feature in enumerate(features_list):
    diff = coef_1[i] - coef_2[i]
    print(f"{feature} | {coef_1[i]:.6f} | {coef_2[i]:.6f} | {diff:.6f}")

```

MNL Model Parameters for Segment 1 (Non-Sat Bookings):

Intercept (β_0): -1.7391

Star Rating (higher = better hotel quality) (β_1): 0.473942

Review Score (higher = better guest reviews) (β_2): 0.109629

Brand Flag (1 = branded hotel, 0 = independent) (β_3): 0.108720

Location Score (higher = better location) (β_4): 0.016666

Accessibility Score (higher = better accessibility) (β_5): 0.052382

Log Historical Price (β_6): -0.070328

Price in USD (β_7): -1.568860

Promotion Flag (1 = has promotion, 0 = no promotion) (β_8): 0.147910

MNL Model Parameters for Segment 2 (Sat Bookings):

Intercept (β_0): -1.7551

Star Rating (higher = better hotel quality) (β_1): 0.358241

Review Score (higher = better guest reviews) (β_2): 0.111053

Brand Flag (1 = branded hotel, 0 = independent) (β_3): 0.098476

Location Score (higher = better location) (β_4): 0.025599

Accessibility Score (higher = better accessibility) (β_5): 0.036260

Log Historical Price (β_6): -0.061769

Price in USD (β_7): -1.155128

Promotion Flag (1 = has promotion, 0 = no promotion) (β_8): 0.167791

Comparison of Coefficients between Segments:

Feature	Non-Sat Booking (β)	Sat Booking (β)	Difference
---------	-----------------------------	-------------------------	------------

prop_starrating	0.473942	0.358241	0.115701
-----------------	----------	----------	----------

prop_review_score	0.109629	0.111053	-0.001424
-------------------	----------	----------	-----------

prop_brand_bool	0.108720	0.098476	0.010244
-----------------	----------	----------	----------

prop_location_score	0.016666	0.025599	-0.008933
---------------------	----------	----------	-----------

prop_accessibility_score	0.052382	0.036260	0.016122
--------------------------	----------	----------	----------

prop_log_historical_price	-0.070328	-0.061769	-0.008559
---------------------------	-----------	-----------	-----------

price_usd	-1.568860	-1.155128	-0.413732
-----------	-----------	-----------	-----------

promotion_flag	0.147910	0.167791	-0.019882
----------------	----------	----------	-----------

Question 5

```
In [1]: #Using smth means: pre-compile and load smth as a package.
using Pkg;
#Set the Gurobi path
ENV["GUROBI_HOME"] = "C:\\gurobi1103\\win64"
#For first timers (comment after): compile Gurobi with the given path
Pkg.build("Gurobi")
#Add the packages that may not be installed.
Pkg.add(["Latexify", "Polyhedra", "Plots", "Makie", "CDDLib", "JuMP", "Gurobi", "Ab
#Load the packages
using JuMP, Latexify, Gurobi, LinearAlgebra, Polyhedra, CDDLib, Plots, SparseArrays

display("Initialization done! All set :-)")
```

```
Building Gurobi → `C:\Users\user\.julia\scratchspaces\44cfe95a-1eb2-52ea-b672-e2
afdf69b78f\53cc56f49295c6b41da670fcf0e0cd812c9580c7\build.log`
Resolving package versions...
No Changes to `C:\Users\user\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\user\.julia\environments\v1.11\Manifest.toml`
Warning: Everything from AbstractPlotting.jl got moved to Makie.jl.
Have a look at the README for information on how to upgrade:
https://github.com/JuliaPlots/AbstractPlotting.jl
@ AbstractPlotting C:\Users\user\.julia\packages\AbstractPlotting\2A5iv\src\Abstra
ctPlotting.jl:261
"Initialization done! All set :-)"
```

```
In [2]: Pkg.status("Gurobi")
```

```
Status `C:\Users\user\.julia\environments\v1.11\Project.toml`
^ [2e9cd046] Gurobi v1.3.1
Info Packages marked with ^ have new versions available and may be upgradable.
```

```
In [3]: using Pkg
Pkg.add("CSV")
Pkg.add("DataFrames")
Pkg.add("Plots")
```

```
Resolving package versions...
No Changes to `C:\Users\user\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\user\.julia\environments\v1.11\Manifest.toml`
Resolving package versions...
No Changes to `C:\Users\user\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\user\.julia\environments\v1.11\Manifest.toml`
Resolving package versions...
No Changes to `C:\Users\user\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\user\.julia\environments\v1.11\Manifest.toml`
```

```
In [41]: # Model parameters
```

```
thetas = [0.5431, 0.4569]
intercepts = [-1.9180, -1.5397] #  $\theta_0$  for Segment 1 and Segment 2
```

```

#  $\theta_1$  to  $\theta_8$  for each segment (rows = segments, columns = features)
betas = [
    0.379038    0.124630    0.092098   -0.021459    0.057844   -0.094884   -1.071555    0.1
    0.462759    0.095586    0.112906    0.087280    0.025559   -0.031981   -1.696923    0.1
]

# Datasets

using CSV, DataFrames
# Read CSV file into DataFrame
data1 = CSV.read("data1.csv", DataFrame)
data2 = CSV.read("data2.csv", DataFrame)
data3 = CSV.read("data3.csv", DataFrame)
data4 = CSV.read("data4.csv", DataFrame)
df_norm_params = CSV.read("normalization_params_q1.csv", DataFrame)

data1_original = deepcopy(data1)
data2_original = deepcopy(data2)
data3_original = deepcopy(data3)
data4_original = deepcopy(data4);

```

```

In [42]: # Loop through each column of data1,2,3,4 and normalize using df_norm_params
for col in names(data1)
    μ = df_norm_params[1, col] # mean
    σ = df_norm_params[2, col] # std
    data1[:, col] .= (data1[:, col] .- μ) ./ σ
end

for col in names(data2)
    μ = df_norm_params[1, col] # mean
    σ = df_norm_params[2, col] # std
    data2[:, col] .= (data2[:, col] .- μ) ./ σ
end

for col in names(data3)
    μ = df_norm_params[1, col] # mean
    σ = df_norm_params[2, col] # std
    data3[:, col] .= (data3[:, col] .- μ) ./ σ
end

for col in names(data4)
    μ = df_norm_params[1, col] # mean
    σ = df_norm_params[2, col] # std
    data4[:, col] .= (data4[:, col] .- μ) ./ σ
end

```

data1

```

In [43]: # Cleaning up data1, to repeat for data2, 3, 4
using DataFrames, LinearAlgebra

# Step 1: Model parameters (above)

# Step 2: Extract features from DataFrame

```

```

features = Matrix(data1)

# Step 3: Compute  $v[i, k] = \exp(\theta_{0k} + \sum_j \theta_{jk} x_{ji})$ 
n = size(features, 1)
K = size(betas, 1)
v = Array{Float64}(undef, n, K)

for k in 1:K
    for i in 1:n
        utility = intercepts[k] + dot(betas[k, :], features[i, :])
        v[i, k] = exp(utility)
    end
end

# Price matrix
p = data1_original[:, :price_usd];

```

Model

do not know customer segment type

$$\begin{aligned}
 \max \quad & \sum_{k=1}^2 \theta_k z_k \\
 \text{s.t.} \quad & -My_i \leq x_{ik} \leq My_i, & \forall i, \forall k \\
 & -M(1-y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M(1-y_i) & \forall i, \forall k \\
 & z_k \leq \sum_{i=1}^n x_{ik} v_{ik} & \forall k \\
 & y_i \in \{0, 1\} & \forall i \\
 & z_k \geq 0 & \forall k
 \end{aligned}$$

```

In [44]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_segment_k = 2
num_hotels_i = nrow(data1)

#create variables
@variable(model, z[1:num_segment_k] >= 0)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i, 1:num_segment_k])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, sum(thetas[k] * z[k] for k in 1:num_segment_k))

# Set up constraints

```

```

# Constraint 1:  $-M y_i \leq x_{ik} \leq M y_i$  for all  $i, k$ 
M = 1000
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * y[i] <= x[i, k])
    @constraint(model, x[i, k] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M (1 - y_i)$  for all  $i, k$ 
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * (1 - y[i]) + p[i] - z[k] <= x[i, k])
    @constraint(model, x[i, k] <= p[i] - z[k] + M * (1 - y[i]))
end

# Constraint 3:  $z_k \leq \text{Sum over } i \text{ of } x_{ik} v_{ik}$  for all  $k$ 
for k in 1:num_segment_k
    @constraint(model, z[k] <= sum(x[i, k] * v[i, k] for i in 1:num_hotels_i))
end

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```

In [45]: #optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("data1 without knowing customer segment type")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))

selected_hotels = []

for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))

data1_S_rev = objective_value(model)
data1_S = copy(selected_hotels);

```

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

data1 without knowing customer segment type

Optimal Objective Function value (Exp Revenue): 107.30479932507615

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 6, 12, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26

Model

know customer segment type

$$\begin{aligned}
& \max z \\
& \text{s.t.} \quad -My_i \leq x_i \leq My_i, & \forall i \\
& \quad \quad -M(1-y_i) + p_i - z \leq x_i \leq p_i - z + M(1-y_i) & \forall i \\
& \quad \quad z \leq \sum_{i=1}^n x_i v_i \\
& \quad \quad y_i \in \{0, 1\} & \forall i
\end{aligned}$$

```
In [46]: # Extract the two columns of v representing the two segments
v_1 = v[:, 1]
v_2 = v[:, 2];
```

```
In [47]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data1)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, z)

# Set up constraints

# Constraint 1: -M y_i <= x_i <= M y_i for all i
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2: -M (1-y_i) + p_i - z <= x_i <= p_i - z + M (1 - y_i) for all i
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3: z <= Sum over i of x_i v_i
@constraint(model, z <= sum(x[i] * v_1[i] for i in 1:num_hotels_i));
```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```
In [48]: #optimize the model
optimize!(model)

#get the status of the optimization
```

```

#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S1 of data1")

println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))

selected_hotels = []

for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data1_S1_rev = objective_value(model)
data1_S1 = copy(selected_hotels);

```

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S1 of data1

Optimal Objective Function value (Exp Revenue): 103.47160468652824

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 6, 9, 12, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 26

```

In [49]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data1)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])

```

```

        @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
    end

    # Constraint 3: z <= Sum over i of x_i v_i
    @constraint(model, z <= sum(x[i] * v_2[i] for i in 1:num_hotels_i))

    #optimize the model
    optimize!(model)

    #get the status of the optimization
    #either
    println("Status = ", raw_status(model))

    #print the solution: value and variables
    println("S2 of data1")
    println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
    selected_hotels = []

    for i in 1:num_hotels_i
        if value(y[i]) == 1
            push!(selected_hotels, string(i - 1))
        end
    end
    println("Hotels selected (indices): ", join(selected_hotels, ", "))
    data1_S2_rev = objective_value(model)
    data1_S2 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S2 of data1

Optimal Objective Function value (Exp Revenue): 112.22213523649923

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 6, 15, 17, 18, 19, 20, 21, 22, 23, 24

data2

```

In [50]: # Cleaning up data2
using DataFrames, LinearAlgebra

# Step 1: Model parameters (done above)

# Step 2: Extract features from DataFrame
features = Matrix(data2)

# Step 3: Compute  $v[i, k] = \exp(\theta_{0k} + \sum_j \theta_{jk} x_{ji})$ 
n = size(features, 1)
K = size(betas, 1)
v = Array{Float64}(undef, n, K)

for k in 1:K
    for i in 1:n
        utility = intercepts[k] + dot(betas[k, :], features[i, :])
        v[i, k] = exp(utility)
    end
end

```

```

end

# Price matrix
p = data2_original[:, :price_usd];

```

data2 - do not know customer segment type

```

In [51]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_segment_k = 2
num_hotels_i = nrow(data2)

#create variables
@variable(model, z[1:num_segment_k] >= 0)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i, 1:num_segment_k])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, sum(thetas[k] * z[k] for k in 1:num_segment_k))

# Set up constraints

# Constraint 1:  $-M y_i \leq x_{ik} \leq M y_i$  for all  $i, k$ 
M = maximum(p) + 10
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * y[i] <= x[i, k])
    @constraint(model, x[i, k] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * (1 - y[i]) + p[i] - z[k] <= x[i, k])
    @constraint(model, x[i, k] <= p[i] - z[k] + M * (1 - y[i]))
end

# Constraint 3:  $z_k \leq \text{Sum over } i \text{ of } x_{ik} v_{ik}$  for all  $k$ 
for k in 1:num_segment_k
    @constraint(model, z[k] <= sum(x[i, k] * v[i, k] for i in 1:num_hotels_i))
end

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```

In [52]: # Sort by price
sorted_indices = sortperm(data2[:, :price_usd], rev=true) # highest price first

# Mark top 5 indices
top5_indices = sorted_indices[1:5]

```

```

# Set warm start values (without modifying data2)
for i in 1:num_hotels_i
    if i in top5_indices
        set_start_value(y[i], 1)
    else
        set_start_value(y[i], 0)
    end
end

set_optimizer_attribute(model, "TimeLimit", 180)      # Stop after 180 seconds
set_optimizer_attribute(model, "Presolve", 2)        # Use aggressive presolve
set_optimizer_attribute(model, "Cuts", 2)            # Use aggressive cuts

# Optimize the model
optimize!(model)

# Get status of optimization
status = termination_status(model)
raw = raw_status(model)
println("Status = ", raw)

# Print results based on status
println("data2 without knowing customer segment type")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data2_S_rev = objective_value(model)
data2_S = copy(selected_hotels);

```

Set parameter Cuts to value 2

Set parameter TimeLimit to value 180

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

data2 without knowing customer segment type

Optimal Objective Function value (Exp Revenue): 131.2791280217877

Hotels selected (indices): 0, 1, 6, 7, 8, 9, 10, 21, 23, 25

data2 - know customer segment type

S1

```

In [53]: # Extract the two columns of v representing the two segments
v_1 = v[:, 1]
v_2 = v[:, 2]

#Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)

```

```

set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data2)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_1[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S1 of data2")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data2_S1_rev = objective_value(model)
data2_S1 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S1 of data2

Optimal Objective Function value (Exp Revenue): 126.92370023226118

Hotels selected (indices): 0, 1, 6, 7, 8, 9, 10, 21, 23, 25

S2

```
In [54]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data2)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_2[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S2 of data2")
```

```
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data2_S2_rev = objective_value(model)
data2_S2 = copy(selected_hotels);
```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S2 of data2

Optimal Objective Function value (Exp Revenue): 137.37465667244123

Hotels selected (indices): 0, 1, 6, 7, 8, 10, 21

data3

```
In [55]: # Cleaning up data3
using DataFrames, LinearAlgebra

# Step 1: Model parameters (done above)

# Step 2: Extract features from DataFrame
features = Matrix(data3)

# Step 3: Compute  $v[i, k] = \exp(\theta_{0k} + \sum_j \theta_{jk} x_{ji})$ 
n = size(features, 1)
K = size(betas, 1)
v = Array{Float64}(undef, n, K)

for k in 1:K
    for i in 1:n
        utility = intercepts[k] + dot(betas[k, :], features[i, :])
        v[i, k] = exp(utility)
    end
end

# Price matrix
p = data3_original[:, :price_usd];
```

data3 - do not know customer segment type

```
In [56]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
```

```

num_segment_k = 2
num_hotels_i = nrow(data3)

#create variables
@variable(model, z[1:num_segment_k] >= 0)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i, 1:num_segment_k])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, sum(thetas[k] * z[k] for k in 1:num_segment_k))

# Set up constraints

# Constraint 1:  $-M y_i \leq x_{ik} \leq M y_i$  for all  $i, k$ 
M = maximum(p) + 10
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * y[i] <= x[i, k])
    @constraint(model, x[i, k] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M (1 - y_i)$  for all  $i, k$ 
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * (1 - y[i]) + p[i] - z[k] <= x[i, k])
    @constraint(model, x[i, k] <= p[i] - z[k] + M * (1 - y[i]))
end

# Constraint 3:  $z_k \leq \text{Sum over } i \text{ of } x_{ik} v_{ik}$  for all  $k$ 
for k in 1:num_segment_k
    @constraint(model, z[k] <= sum(x[i, k] * v[i, k] for i in 1:num_hotels_i))
end

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```

In [57]: # Sort by price
sorted_indices = sortperm(data3[:, :price_usd], rev=true) # highest price first

# Mark top 5 indices
top5_indices = sorted_indices[1:5]

# Set warm start values (without modifying data2)
for i in 1:num_hotels_i
    if i in top5_indices
        set_start_value(y[i], 1)
    else
        set_start_value(y[i], 0)
    end
end

set_optimizer_attribute(model, "TimeLimit", 180) # Stop after 180 seconds
set_optimizer_attribute(model, "Presolve", 2) # Use aggressive presolve
set_optimizer_attribute(model, "Cuts", 2) # Use aggressive cuts

# Optimize the model
optimize!(model)

```

```

# Get status of optimization
status = termination_status(model)
raw = raw_status(model)
println("Status = ", raw)

# Print results based on status
println("data3 without knowing customer segment type")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data3_S_rev = objective_value(model)
data3_S = copy(selected_hotels);

```

Set parameter Cuts to value 2

Set parameter TimeLimit to value 180

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

data3 without knowing customer segment type

Optimal Objective Function value (Exp Revenue): 120.99824527110692

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24

data3 - know customer segment type

S1

```

In [58]: # Extract the two columns of v representing the two segments
v_1 = v[:, 1]
v_2 = v[:, 2]

#Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data3)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, z)

# Set up constraints

```

```

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_1[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S1 of data3")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data3_S1_rev = objective_value(model)
data3_S1 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S1 of data3

Optimal Objective Function value (Exp Revenue): 117.52114848843323

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24, 25

S2

In [59]: *#Building the JuMP model*
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}
set_optimizer_attribute(model, "OutputFlag", 0)

Define the variables
num_hotels_i = nrow(data3)

```

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_2[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S2 of data3")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data3_S2_rev = objective_value(model)
data3_S2 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S2 of data3

Optimal Objective Function value (Exp Revenue): 125.34894655852843

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 19, 23, 2

4

data4

```
In [60]: # Cleaning up data4
using DataFrames, LinearAlgebra

# Step 1: Model parameters (done above)

# Step 2: Extract features from DataFrame
features = Matrix{Float64}(data4)

# Step 3: Compute  $v[i, k] = \exp(\theta_{0k} + \sum_j \theta_{jk} x_{ji})$ 
n = size(features, 1)
K = size(betas, 1)
v = Array{Float64}(undef, n, K)

for k in 1:K
    for i in 1:n
        utility = intercepts[k] + dot(betas[k, :], features[i, :])
        v[i, k] = exp(utility)
    end
end

# Price matrix
p = data4_original[:, :price_usd];
```

data4 - do not know customer segment type

```
In [61]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}()
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_segment_k = 2
num_hotels_i = nrow(data4)

#create variables
@variable(model, z[1:num_segment_k] >= 0)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i, 1:num_segment_k])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, sum(thetas[k] * z[k] for k in 1:num_segment_k))

# Set up constraints

# Constraint 1:  $-M y_i \leq x_{ik} \leq M y_i$  for all  $i, k$ 
M = maximum(p) + 10
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * y[i] <= x[i, k])
    @constraint(model, x[i, k] <= M * y[i])
end
```

```

end

# Constraint 2:  $-M(1-y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M(1-y_i)$  for all  $i$ 
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * (1 - y[i]) + p[i] - z[k] <= x[i, k])
    @constraint(model, x[i, k] <= p[i] - z[k] + M * (1 - y[i]))
end

# Constraint 3:  $z_k \leq \text{Sum over } i \text{ of } x_{ik} v_{ik}$  for all  $k$ 
for k in 1:num_segment_k
    @constraint(model, z[k] <= sum(x[i, k] * v[i, k] for i in 1:num_hotels_i))
end

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```

In [62]: # Sort by price
sorted_indices = sortperm(data4[:, :price_usd], rev=true) # highest price first

# Mark top 5 indices
top5_indices = sorted_indices[1:5]

# Set warm start values (without modifying data2)
for i in 1:num_hotels_i
    if i in top5_indices
        set_start_value(y[i], 1)
    else
        set_start_value(y[i], 0)
    end
end

set_optimizer_attribute(model, "TimeLimit", 180) # Stop after 180 seconds
set_optimizer_attribute(model, "Presolve", 2) # Use aggressive presolve
set_optimizer_attribute(model, "Cuts", 2) # Use aggressive cuts

# Optimize the model
optimize!(model)

# Get status of optimization
status = termination_status(model)
raw = raw_status(model)
println("Status = ", raw)

# Print results based on status
println("data4 without knowing customer segment type")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data4_S_rev = objective_value(model)
data4_S = copy(selected_hotels);

```

Set parameter Cuts to value 2
Set parameter TimeLimit to value 180
Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.
data4 without knowing customer segment type
Optimal Objective Function value (Exp Revenue): 97.38280634349921
Hotels selected (indices): 3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26

data4 - know customer segment type

S1

```
In [63]: # Extract the two columns of v representing the two segments
v_1 = v[:, 1]
v_2 = v[:, 2]

#Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data4)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all i
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all i
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_1[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)
```

```

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S1 of data4")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data4_S1_rev = objective_value(model)
data4_S1 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S1 of data4

Optimal Objective Function value (Exp Revenue): 94.69566705306295

Hotels selected (indices): 3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26

S2

```

In [64]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data4)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i

```

```

    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3: z <= Sum over i of x_i v_i
@constraint(model, z <= sum(x[i] * v_2[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S2 of data4")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data4_S2_rev = objective_value(model)
data4_S2 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S2 of data4

Optimal Objective Function value (Exp Revenue): 100.63909615263897

Hotels selected (indices): 3, 4, 6, 10, 15, 18, 19, 20, 21, 26

Summary of all revenues and hotels selected

```

In [65]: println(join(data1_S, ", "))
println(join(data1_S1, ", "))
println(join(data1_S2, ", "))

println(join(data2_S, ", "))
println(join(data2_S1, ", "))
println(join(data2_S2, ", "))

println(join(data3_S, ", "))
println(join(data3_S1, ", "))
println(join(data3_S2, ", "))

println(join(data4_S, ", "))
println(join(data4_S1, ", "))
println(join(data4_S2, ", "))

```

```

0, 1, 2, 3, 4, 5, 6, 12, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26
0, 1, 2, 3, 4, 5, 6, 9, 12, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 26
0, 1, 2, 3, 4, 5, 6, 15, 17, 18, 19, 20, 21, 22, 23, 24
0, 1, 6, 7, 8, 9, 10, 21, 23, 25
0, 1, 6, 7, 8, 9, 10, 21, 23, 25
0, 1, 6, 7, 8, 10, 21
0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24
0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24, 25
0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 19, 23, 24
3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26
3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26
3, 4, 6, 10, 15, 18, 19, 20, 21, 26

```

In [66]: using Printf

```

@printf("%.2f\n", data1_S_rev)
@printf("%.2f\n", data1_S1_rev)
@printf("%.2f\n", data1_S2_rev)

@printf("%.2f\n", data2_S_rev)
@printf("%.2f\n", data2_S1_rev)
@printf("%.2f\n", data2_S2_rev)

@printf("%.2f\n", data3_S_rev)
@printf("%.2f\n", data3_S1_rev)
@printf("%.2f\n", data3_S2_rev)

@printf("%.2f\n", data4_S_rev)
@printf("%.2f\n", data4_S1_rev)
@printf("%.2f\n", data4_S2_rev)

```

```

107.30
103.47
112.22
131.28
126.92
137.37
121.00
117.52
125.35
97.38
94.70
100.64

```

Question 6

```
In [1]: #Using smth means: pre-compile and load smth as a package.
using Pkg;
#Set the Gurobi path
ENV["GUROBI_HOME"] = "C:\\gurobi1103\\win64"
#For first timers (comment after): compile Gurobi with the given path
Pkg.build("Gurobi")
#Add the packages that may not be installed.
Pkg.add(["Latexify", "Polyhedra", "Plots", "Makie", "CDDLib", "JuMP", "Gurobi", "AbstractPlotting"])
#Load the packages
using JuMP, Latexify, Gurobi, LinearAlgebra, Polyhedra, CDDLib, Plots, SparseArrays, Makie, AbstractPlotting

display("Initialization done! All set :-)")
```

```
Building Gurobi → `C:\Users\user\.julia\scratchspaces\44cfe95a-1eb2-52ea-b672-e2afdf69b78f\53cc56f49295c6b41da670fcf0e0cd812c9580c7\build.log`
Resolving package versions...
No Changes to `C:\Users\user\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\user\.julia\environments\v1.11\Manifest.toml`
Warning: Everything from AbstractPlotting.jl got moved to Makie.jl.
Have a look at the README for information on how to upgrade:
https://github.com/JuliaPlots/AbstractPlotting.jl
@ AbstractPlotting C:\Users\user\.julia\packages\AbstractPlotting\2A5iv\src\AbstractPlotting.jl:261
"Initialization done! All set :-)"
```

```
In [2]: Pkg.status("Gurobi")

Status `C:\Users\user\.julia\environments\v1.11\Project.toml`
^ [2e9cd046] Gurobi v1.3.1
Info Packages marked with ^ have new versions available and may be upgradable.
```

```
In [3]: using Pkg
Pkg.add("CSV")
Pkg.add("DataFrames")
Pkg.add("Plots")

Resolving package versions...
No Changes to `C:\Users\user\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\user\.julia\environments\v1.11\Manifest.toml`
Resolving package versions...
No Changes to `C:\Users\user\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\user\.julia\environments\v1.11\Manifest.toml`
Resolving package versions...
No Changes to `C:\Users\user\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\user\.julia\environments\v1.11\Manifest.toml`
```

```
In [4]: # Model parameters

thetas = [0.4234, 0.5766]

# Updated intercepts ( $\theta_0$  for Segment 1 and Segment 2)
intercepts = [-1.7391, -1.7551] # Non-Sat, Sat

# Updated betas: rows = segments, columns = features ( $\theta_1$  to  $\theta_8$ )
betas = [
    0.473942  0.109629  0.108720  0.016666  0.052382 -0.070328 -1.568860  0.147910; # Segment 1 (Non-
    0.358241  0.111053  0.098476  0.025599  0.036260 -0.061769 -1.155128  0.167791 # Segment 2 (Sat,
]

# Datasets

using CSV, DataFrames
# Read CSV file into DataFrame
data1 = CSV.read("data1.csv", DataFrame)
data2 = CSV.read("data2.csv", DataFrame)
data3 = CSV.read("data3.csv", DataFrame)
data4 = CSV.read("data4.csv", DataFrame)
df_norm_params = CSV.read("normalization_params_q1.csv", DataFrame)

data1_original = deepcopy(data1)
data2_original = deepcopy(data2)
data3_original = deepcopy(data3)
data4_original = deepcopy(data4);
```

```
In [5]: # Loop through each column of data1,2,3,4 and normalize using df_norm_params
for col in names(data1)
     $\mu$  = df_norm_params[1, col] # mean
     $\sigma$  = df_norm_params[2, col] # std
    data1[:, col] .= (data1[:, col] .-  $\mu$ ) ./  $\sigma$ 
end

for col in names(data2)
     $\mu$  = df_norm_params[1, col] # mean
     $\sigma$  = df_norm_params[2, col] # std
    data2[:, col] .= (data2[:, col] .-  $\mu$ ) ./  $\sigma$ 
end

for col in names(data3)
     $\mu$  = df_norm_params[1, col] # mean
     $\sigma$  = df_norm_params[2, col] # std
    data3[:, col] .= (data3[:, col] .-  $\mu$ ) ./  $\sigma$ 
end

for col in names(data4)
     $\mu$  = df_norm_params[1, col] # mean
     $\sigma$  = df_norm_params[2, col] # std
    data4[:, col] .= (data4[:, col] .-  $\mu$ ) ./  $\sigma$ 
end
```

data1

```
In [6]: # Cleaning up data1, to repeat for data2, 3, 4
using DataFrames, LinearAlgebra

# Step 1: Model parameters (above)

# Step 2: Extract features from DataFrame
features = Matrix{Float64}(data1)

# Step 3: Compute  $v[i, k] = \exp(\theta_{0k} + \sum_j \theta_{jk} x_{ji})$ 
n = size(features, 1)
K = size(betas, 1)
v = Array{Float64}(undef, n, K)

for k in 1:K
    for i in 1:n
        utility = intercepts[k] + dot(betas[k, :], features[i, :])
        v[i, k] = exp(utility)
    end
end
```

```
# Price matrix
p = data1_original[:, :price_usd];
```

Model

do not know customer segment type

$$\begin{aligned} \max \quad & \sum_{k=1}^2 \theta_k z_k \\ \text{s.t.} \quad & -M y_i \leq x_{ik} \leq M y_i, \quad \forall i, k \\ & -M(1 - y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M(1 - y_i), \quad \forall i, k \\ & z_k \leq \sum_{i=1}^n x_{ik} v_{ik}, \quad \forall k \\ & y_i \in \{0, 1\}, \quad \forall i \\ & z_k \geq 0, \quad \forall k \end{aligned}$$

```
In [7]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_segment_k = 2
num_hotels_i = nrow(data1)

#create variables
@variable(model, z[1:num_segment_k] >= 0)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i, 1:num_segment_k])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, sum(thetas[k] * z[k] for k in 1:num_segment_k))

# Set up constraints

# Constraint 1: -M y_i <= x_ik <= M y_i for all i, k
M = 1000
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * y[i] <= x[i, k])
    @constraint(model, x[i, k] <= M * y[i])
end

# Constraint 2: -M (1-y_i) + p_i - z_k <= x_ik <= p_i - z_k + M (1 - y_i) for all i, k
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * (1 - y[i]) + p[i] - z[k] <= x[i, k])
    @constraint(model, x[i, k] <= p[i] - z[k] + M * (1 - y[i]))
end

# Constraint 3: z_k <= Sum over i of x_ik v_ik for all k
for k in 1:num_segment_k
    @constraint(model, z[k] <= sum(x[i, k] * v[i, k] for i in 1:num_hotels_i))
end
```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```
In [8]: #optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("data1 without knowing customer segment type")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))

selected_hotels = []

for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
```

```
data1_S_rev = objective_value(model)
data1_S = copy(selected_hotels);
```

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.
data1 without knowing customer segment type
Optimal Objective Function value (Exp Revenue): 107.35217363043266
Hotels selected (indices): 0, 1, 2, 3, 4, 5, 6, 12, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26

Model

know customer segment type

$$\begin{aligned} \max z \quad & -M y_i \leq x_i \leq M y_i, \quad \forall i \quad \& \quad -M(1 - y_i) + p_i - z \leq x_i \\ & \leq p_i - z + M(1 - y_i) \quad \forall i \quad \& \quad z \leq \sum_{i=1}^n x_i v_i \quad \& \quad y_i \in \{0, 1\} \quad \forall i \end{aligned}$$

```
In [9]: # Extract the two columns of v representing the two segments
v_1 = v[:, 1]
v_2 = v[:, 2];
```

```
In [10]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data1)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, z)

# Set up constraints

# Constraint 1: -M y_i <= x_i <= M y_i for all i
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2: -M (1-y_i) + p_i - z <= x_i <= p_i - z + M (1 - y_i) for all i
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3: z <= Sum over i of x_i v_i
@constraint(model, z <= sum(x[i] * v_1[i] for i in 1:num_hotels_i));
```

Set parameter Username
Academic license - for non-commercial use only - expires 2025-10-20

```
In [11]: #optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S1 of data1")

println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))

selected_hotels = []

for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
```

```

end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data1_S1_rev = objective_value(model)
data1_S1 = copy(selected_hotels);

```

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.
 S1 of data1
 Optimal Objective Function value (Exp Revenue): 108.09118841600318
 Hotels selected (indices): 0, 1, 2, 3, 4, 5, 6, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26

```

In [12]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data1)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, z)

# Set up constraints

# Constraint 1: -M y_i <= x_i <= M y_i for all i
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2: -M (1-y_i) + p_i - z <= x_i <= p_i - z + M (1 - y_i) for all i
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3: z <= Sum over i of x_i v_i
@constraint(model, z <= sum(x[i] * v_2[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S2 of data1")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []

for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data1_S2_rev = objective_value(model)
data1_S2 = copy(selected_hotels);

```

Set parameter Username
 Academic license - for non-commercial use only - expires 2025-10-20
 Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.
 S2 of data1
 Optimal Objective Function value (Exp Revenue): 106.82050963879034
 Hotels selected (indices): 0, 1, 2, 3, 4, 5, 6, 9, 12, 14, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26

data2

```
In [13]: # Cleaning up data2
using DataFrames, LinearAlgebra

# Step 1: Model parameters (done above)

# Step 2: Extract features from DataFrame
features = Matrix{Float64}(data2)

# Step 3: Compute  $v[i, k] = \exp(\theta_{0k} + \sum_j \theta_{jk} x_{ji})$ 
n = size(features, 1)
K = size(betas, 1)
v = Array{Float64}(undef, n, K)

for k in 1:K
    for i in 1:n
        utility = intercepts[k] + dot(betas[k, :], features[i, :])
        v[i, k] = exp(utility)
    end
end

# Price matrix
p = data2_original[:, :price_usd];
```

data2 - do not know customer segment type

```
In [14]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_segment_k = 2
num_hotels_i = nrow(data2)

#create variables
@variable(model, z[1:num_segment_k] >= 0)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i, 1:num_segment_k])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, sum(thetas[k] * z[k] for k in 1:num_segment_k))

# Set up constraints

# Constraint 1:  $-M y_i \leq x_{ik} \leq M y_i$  for all  $i, k$ 
M = maximum(p) + 10
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * y[i] <= x[i, k])
    @constraint(model, x[i, k] <= M * y[i])
end

# Constraint 2:  $-M (1 - y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M (1 - y_i)$  for all  $i, k$ 
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * (1 - y[i]) + p[i] - z[k] <= x[i, k])
    @constraint(model, x[i, k] <= p[i] - z[k] + M * (1 - y[i]))
end

# Constraint 3:  $z_k \leq \text{Sum over } i \text{ of } x_{ik} v_{ik}$  for all  $k$ 
for k in 1:num_segment_k
    @constraint(model, z[k] <= sum(x[i, k] * v[i, k] for i in 1:num_hotels_i))
end
```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```
In [15]: # Sort by price
sorted_indices = sortperm(data2[:, :price_usd], rev=true) # highest price first

# Mark top 5 indices
top5_indices = sorted_indices[1:5]

# Set warm start values (without modifying data2)
for i in 1:num_hotels_i
```

```

    if i in top5_indices
        set_start_value(y[i], 1)
    else
        set_start_value(y[i], 0)
    end
end

set_optimizer_attribute(model, "TimeLimit", 180)      # Stop after 180 seconds
set_optimizer_attribute(model, "Presolve", 2)         # Use aggressive presolve
set_optimizer_attribute(model, "Cuts", 2)             # Use aggressive cuts

# Optimize the model
optimize!(model)

# Get status of optimization
status = termination_status(model)
raw = raw_status(model)
println("Status = ", raw)

# Print results based on status
println("data2 without knowing customer segment type")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data2_S_rev = objective_value(model)
data2_S = copy(selected_hotels);

```

Set parameter Cuts to value 2

Set parameter TimeLimit to value 180

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

data2 without knowing customer segment type

Optimal Objective Function value (Exp Revenue): 130.94868992632198

Hotels selected (indices): 0, 1, 6, 7, 8, 9, 10, 21, 23, 25

data2 - know customer segment type

S1

```

In [16]: # Extract the two columns of v representing the two segments
v_1 = v[:, 1]
v_2 = v[:, 2]

#Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data2)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1 - y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i

```

```

        @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
        @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
    end

    # Constraint 3: z <= Sum over i of x_i v_i
    @constraint(model, z <= sum(x[i] * v_1[i] for i in 1:num_hotels_i))

    #optimize the model
    optimize!(model)

    #get the status of the optimization
    #either
    println("Status = ", raw_status(model))

    #print the solution: value and variables
    println("S1 of data2")
    println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
    selected_hotels = []
    for i in 1:num_hotels_i
        if value(y[i]) == 1
            push!(selected_hotels, string(i - 1))
        end
    end
    println("Hotels selected (indices): ", join(selected_hotels, ", "))
    data2_S1_rev = objective_value(model)
    data2_S1 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S1 of data2

Optimal Objective Function value (Exp Revenue): 132.4592170260214

Hotels selected (indices): 0, 1, 6, 7, 8, 9, 10, 21, 23, 25

S2

```

In [17]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data2)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, z)

# Set up constraints

# Constraint 1: -M y_i <= x_i <= M y_i for all i
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2: -M (1-y_i) + p_i - z <= x_i <= p_i - z + M (1 - y_i) for all i
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3: z <= Sum over i of x_i v_i
@constraint(model, z <= sum(x[i] * v_2[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either

```

```
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S2 of data2")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data2_S2_rev = objective_value(model)
data2_S2 = copy(selected_hotels);
```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S2 of data2

Optimal Objective Function value (Exp Revenue): 129.83950301336222

Hotels selected (indices): 0, 1, 6, 7, 8, 9, 10, 21, 23, 25

data3

```
In [18]: # Cleaning up data3
using DataFrames, LinearAlgebra

# Step 1: Model parameters (done above)

# Step 2: Extract features from DataFrame
features = Matrix{Float64}(data3)

# Step 3: Compute  $v[i, k] = \exp(\theta_{0k} + \sum_j \theta_{jk} x_{ji})$ 
n = size(features, 1)
K = size(betas, 1)
v = Array{Float64}(undef, n, K)

for k in 1:K
    for i in 1:n
        utility = intercepts[k] + dot(betas[k, :], features[i, :])
        v[i, k] = exp(utility)
    end
end

# Price matrix
p = data3_original[:, :price_usd];
```

data3 - do not know customer segment type

```
In [19]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_segment_k = 2
num_hotels_i = nrow(data3)

#create variables
@variable(model, z[1:num_segment_k] >= 0)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i, 1:num_segment_k])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, sum(thetas[k] * z[k] for k in 1:num_segment_k))

# Set up constraints

# Constraint 1:  $-M y_i \leq x_{ik} \leq M y_i$  for all  $i, k$ 
M = maximum(p) + 10
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * y[i] <= x[i, k])
    @constraint(model, x[i, k] <= M * y[i])
end
```

```

end

# Constraint 2:  $-M(1-y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M(1-y_i)$  for all  $i, k$ 
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * (1 - y[i]) + p[i] - z[k] <= x[i, k])
    @constraint(model, x[i, k] <= p[i] - z[k] + M * (1 - y[i]))
end

# Constraint 3:  $z_k \leq \text{Sum over } i \text{ of } x_{ik} v_{ik}$  for all  $k$ 
for k in 1:num_segment_k
    @constraint(model, z[k] <= sum(x[i, k] * v[i, k] for i in 1:num_hotels_i))
end

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```

In [20]: # Sort by price
sorted_indices = sortperm(data3[:, :price_usd], rev=true) # highest price first

# Mark top 5 indices
top5_indices = sorted_indices[1:5]

# Set warm start values (without modifying data2)
for i in 1:num_hotels_i
    if i in top5_indices
        set_start_value(y[i], 1)
    else
        set_start_value(y[i], 0)
    end
end

set_optimizer_attribute(model, "Timelimit", 180) # Stop after 180 seconds
set_optimizer_attribute(model, "Presolve", 2) # Use aggressive presolve
set_optimizer_attribute(model, "Cuts", 2) # Use aggressive cuts

# Optimize the model
optimize!(model)

# Get status of optimization
status = termination_status(model)
raw = raw_status(model)
println("Status = ", raw)

# Print results based on status
println("data3 without knowing customer segment type")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data3_S_rev = objective_value(model)
data3_S = copy(selected_hotels);

```

Set parameter Cuts to value 2

Set parameter Timelimit to value 180

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

data3 without knowing customer segment type

Optimal Objective Function value (Exp Revenue): 121.17314583189967

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24

data3 - know customer segment type

S1

```

In [21]: # Extract the two columns of v representing the two segments
v_1 = v[:, 1]
v_2 = v[:, 2]

#Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model{Gurobi.Optimizer}()
set_optimizer_attribute(model, "OutputFlag", 0)

```

```

# Define the variables
num_hotels_i = nrow(data3)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1 - y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_1[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S1 of data3")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data3_S1_rev = objective_value(model)
data3_S1 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S1 of data3

Optimal Objective Function value (Exp Revenue): 120.59367808510699

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24

S2

```

In [22]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data3)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, z)

```

```

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1 - y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_2[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S2 of data3")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data3_S2_rev = objective_value(model)
data3_S2 = copy(selected_hotels);

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

S2 of data3

Optimal Objective Function value (Exp Revenue): 121.5986516314002

Hotels selected (indices): 0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24

data4

```

In [23]: # Cleaning up data4
using DataFrames, LinearAlgebra

# Step 1: Model parameters (done above)

# Step 2: Extract features from DataFrame
features = Matrix{Float64}(data4)

# Step 3: Compute  $v[i, k] = \exp(\theta_{0k} + \sum_j \theta_{jk} x_{ji})$ 
n = size(features, 1)
K = size(betas, 1)
v = Array{Float64}(undef, n, K)

for k in 1:K
    for i in 1:n
        utility = intercepts[k] + dot(betas[k, :], features[i, :])
        v[i, k] = exp(utility)
    end
end

# Price matrix
p = data4_original[:, :price_usd];

```

data4 - do not know customer segment type

```

In [24]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

```

```

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_segment_k = 2
num_hotels_i = nrow(data4)

#create variables
@variable(model, z[1:num_segment_k] >= 0)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i, 1:num_segment_k])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, sum(thetas[k] * z[k] for k in 1:num_segment_k))

# Set up constraints

# Constraint 1:  $-M y_i \leq x_{ik} \leq M y_i$  for all  $i, k$ 
M = maximum(p) + 10
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * y[i] <= x[i, k])
    @constraint(model, x[i, k] <= M * y[i])
end

# Constraint 2:  $-M (1 - y_i) + p_i - z_k \leq x_{ik} \leq p_i - z_k + M (1 - y_i)$  for all  $i, k$ 
for i in 1:num_hotels_i, k in 1:num_segment_k
    @constraint(model, -M * (1 - y[i]) + p[i] - z[k] <= x[i, k])
    @constraint(model, x[i, k] <= p[i] - z[k] + M * (1 - y[i]))
end

# Constraint 3:  $z_k \leq \text{Sum over } i \text{ of } x_{ik} v_{ik}$  for all  $k$ 
for k in 1:num_segment_k
    @constraint(model, z[k] <= sum(x[i, k] * v[i, k] for i in 1:num_hotels_i))
end

```

Set parameter Username

Academic license - for non-commercial use only - expires 2025-10-20

```

In [25]: # Sort by price
sorted_indices = sortperm(data4[:, :price_usd], rev=true) # highest price first

# Mark top 5 indices
top5_indices = sorted_indices[1:5]

# Set warm start values (without modifying data2)
for i in 1:num_hotels_i
    if i in top5_indices
        set_start_value(y[i], 1)
    else
        set_start_value(y[i], 0)
    end
end

set_optimizer_attribute(model, "Timelimit", 180) # Stop after 180 seconds
set_optimizer_attribute(model, "Presolve", 2) # Use aggressive presolve
set_optimizer_attribute(model, "Cuts", 2) # Use aggressive cuts

# Optimize the model
optimize!(model)

# Get status of optimization
status = termination_status(model)
raw = raw_status(model)
println("Status = ", raw)

# Print results based on status
println("data4 without knowing customer segment type")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))

```

```
data4_S_rev = objective_value(model)
data4_S = copy(selected_hotels);
```

Set parameter Cuts to value 2

Set parameter TimeLimit to value 180

Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.

data4 without knowing customer segment type

Optimal Objective Function value (Exp Revenue): 97.26345071969826

Hotels selected (indices): 3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26

data4 - know customer segment type

S1

```
In [26]: # Extract the two columns of v representing the two segments
v_1 = v[:, 1]
v_2 = v[:, 2]

#Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data4)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1 - y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_1[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S1 of data4")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data4_S1_rev = objective_value(model)
data4_S1 = copy(selected_hotels);
```

Set parameter Username
 Academic license - for non-commercial use only - expires 2025-10-20
 Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.
 S1 of data4
 Optimal Objective Function value (Exp Revenue): 98.16774262859315
 Hotels selected (indices): 3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26

S2

```
In [27]: #Building the JuMP model
using JuMP, Gurobi, LinearAlgebra

#Initialize the model with a given optimizer, say Gurobi
model = Model(Gurobi.Optimizer)
set_optimizer_attribute(model, "OutputFlag", 0)

# Define the variables
num_hotels_i = nrow(data4)

#create variables
@variable(model, z)
@variable(model, y[1:num_hotels_i], Bin)
@variable(model, x[1:num_hotels_i])

#Set the objective. The arguments are the model object, the objective sense and its expression.
@objective(model, Max, z)

# Set up constraints

# Constraint 1:  $-M y_i \leq x_i \leq M y_i$  for all  $i$ 
M = 1000
for i in 1:num_hotels_i
    @constraint(model, -M * y[i] <= x[i])
    @constraint(model, x[i] <= M * y[i])
end

# Constraint 2:  $-M (1-y_i) + p_i - z \leq x_i \leq p_i - z + M (1 - y_i)$  for all  $i$ 
for i in 1:num_hotels_i
    @constraint(model, -M * (1 - y[i]) + p[i] - z <= x[i])
    @constraint(model, x[i] <= p[i] - z + M * (1 - y[i]))
end

# Constraint 3:  $z \leq \text{Sum over } i \text{ of } x_i v_i$ 
@constraint(model, z <= sum(x[i] * v_2[i] for i in 1:num_hotels_i))

#optimize the model
optimize!(model)

#get the status of the optimization
#either
println("Status = ", raw_status(model))

#print the solution: value and variables
println("S2 of data4")
println("Optimal Objective Function value (Exp Revenue): ", objective_value(model))
selected_hotels = []
for i in 1:num_hotels_i
    if value(y[i]) == 1
        push!(selected_hotels, string(i - 1))
    end
end
println("Hotels selected (indices): ", join(selected_hotels, ", "))
data4_S2_rev = objective_value(model)
data4_S2 = copy(selected_hotels);
```

Set parameter Username
 Academic license - for non-commercial use only - expires 2025-10-20
 Status = Model was solved to optimality (subject to tolerances), and an optimal solution is available.
 S2 of data4
 Optimal Objective Function value (Exp Revenue): 96.59942506200478
 Hotels selected (indices): 3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26

Summary of all revenues and hotels selected

```
In [28]: println(join(data1_S, ", "))
println(join(data1_S1, ", "))
```

```
println(join(data1_S2, ", "))

println(join(data2_S, ", "))
println(join(data2_S1, ", "))
println(join(data2_S2, ", "))

println(join(data3_S, ", "))
println(join(data3_S1, ", "))
println(join(data3_S2, ", "))

println(join(data4_S, ", "))
println(join(data4_S1, ", "))
println(join(data4_S2, ", "))
```

```
0, 1, 2, 3, 4, 5, 6, 12, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26
0, 1, 2, 3, 4, 5, 6, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26
0, 1, 2, 3, 4, 5, 6, 9, 12, 14, 15, 17, 18, 19, 20, 21, 22, 23, 24, 26
0, 1, 6, 7, 8, 9, 10, 21, 23, 25
0, 1, 6, 7, 8, 9, 10, 21, 23, 25
0, 1, 6, 7, 8, 9, 10, 21, 23, 25
0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24
0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24
0, 1, 2, 3, 4, 5, 7, 8, 10, 11, 13, 14, 15, 16, 18, 19, 23, 24
3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26
3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26
3, 4, 6, 8, 10, 15, 18, 19, 20, 21, 26
```

In [29]: **using** Printf

```
@printf("%.2f\n", data1_S_rev)
@printf("%.2f\n", data1_S1_rev)
@printf("%.2f\n", data1_S2_rev)

@printf("%.2f\n", data2_S_rev)
@printf("%.2f\n", data2_S1_rev)
@printf("%.2f\n", data2_S2_rev)

@printf("%.2f\n", data3_S_rev)
@printf("%.2f\n", data3_S1_rev)
@printf("%.2f\n", data3_S2_rev)

@printf("%.2f\n", data4_S_rev)
@printf("%.2f\n", data4_S1_rev)
@printf("%.2f\n", data4_S2_rev)
```

```
107.35
108.09
106.82
130.95
132.46
129.84
121.17
120.59
121.60
97.26
98.17
96.60
```